

# TP 2 : Analyse Lexicale (avec Python et expressions régulières)

## Objectifs

Ce TP a pour but de vous initier à l'écriture d'un **analyseur lexical simple** en Python. Vous allez :

- Définir des expressions régulières pour les tokens,
- Utiliser le module `re` pour construire un **lexeur**,
- Générer une **liste de tokens** à partir d'un code source,
- Identifier les erreurs lexicales.

## Partie 1 – Expressions régulières de base

### 1.1 Tâche :

Définissez en Python une expression régulière (regex) pour chacun des tokens suivants :

1. Mots-clés : `if`, `else`, `while`, `return`
2. Identifiants : lettres/underscore suivi de lettres/chiffres/underscores
3. Entiers positifs
4. Opérateurs : `+`, `-`, `*`, `/`, `=`, `==`, `!=`, `<`, `<=`, `>=`
5. Délimiteurs : `(`, `)`, `{`, `}`, `;`

### Exemple de code :

```
import re

TOKEN_REGEX = [
    ('KEYWORD', r'\b(if|else|while|return)\b'),
    ('ID', r'[a-zA-Z_][a-zA-Z0-9_]*'),
    ('NUM', r'[0-9]+'),
    ('OP', r'(|=|!=|<=|>=|+|-|\*|/)'),
    ('DELIM', r'[\(\)\{\};]'),
    ('SKIP', r'[\t\n]+'),
    ('MISMATCH', r'.')
]
```

## Partie 2 – Lexeur en Python

### 2.1 Tâche :

Implémentez une fonction `tokenize(code)` qui reçoit une chaîne de caractères contenant du code source et retourne la liste des tokens reconnus.

### Exemple attendu :

```
code = "if (x == 5) { return x; }"
# Output :
# [('KEYWORD', 'if'), ('DELIM', '('), ('ID', 'x'), ('OP', '=='), ('NUM',
'5'), ...]
```

### Indice :

Utilisez `re.finditer()` pour parcourir le code et appliquer chaque regex.

## Partie 3 – Affichage et erreurs

### 3.1 Afficher les tokens un par un, ligne par ligne

### 3.2 Signaler les erreurs lexicales (MISMATCH) avec position (indice dans la chaîne)

Exemple de sortie :

```
(KEYWORD, 'if')
(DE LIM, '(')
(ID, 'val')
(OP, '==')
(NUM, '42')
(DE LIM, ')')
(ERROR, '@' à position 23)
```

## Partie 4 – Extensions (facultatives)

1. Gérer les chaînes de caractères entre guillemets : "texte"
2. Construire une **table des symboles** (dictionnaire) pour les identifiants rencontrés
3. Afficher les lignes et colonnes (position exacte) des erreurs